Spring Security Guide- Part 1

🔐 Qu’est-ce que Spring Security ?

Spring Security est un **framework de sécurité** pour les applications Java (Spring), qui permet de gérer :

- **l’authentification** (savoir qui est l’utilisateur)
- **l’autorisation** (ce que l’utilisateur peut faire)

Il est basé sur une **chaîne de filtres** (`FilterChain`) qui interceptent chaque requête HTTP pour appliquer les règles de sécurité avant que celle-ci n’atteigne le contrôleur.

[Server] → [Middleware: handles security] → [Endpoints]

There are two things that are done on the middleware level:

- **Authentication**: security mechanism determines WHO the user is.In case a particular API requires authentication and the information in the request is not sufficient or not valid the system responds with the **401** Unauthorized status code.
- **Authorization**: the security mechanism checks WHAT action the user is trying to perform and if the user has the permission or AUTHORITY to do that particular action. If the particular action is not allowed for the user, the system typically responds with the **403** Forbidden status code.

🧠 AuthenticationManager

- C’est le **gestionnaire central de l’authentification**.
- Il délègue le travail aux `AuthenticationProvider` qui vont vérifier les credentials.

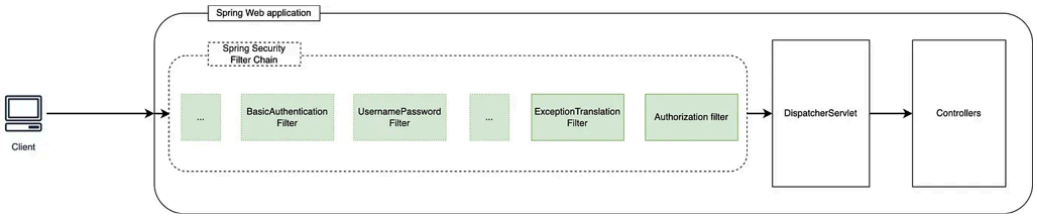
⚙️ AuthenticationProvider

- Implémente la logique d'authentification réelle.
- Par exemple : `DaoAuthenticationProvider` utilise un `UserDetailsService` pour charger l'utilisateur depuis la base de données.

🧾 UserDetailsService

- C’est une interface utilisée pour **charger un utilisateur** depuis la base de données ou toute autre source

[Servlet container] → [Filter] → [Filter]→ [Filter]→ [Filter]→ [Filter]→ [Filter]→ [Filter]→ [Servlet]



Default Spring Security filter chain consists of many filters, but their purpose can be divided into a couple of categories:

- **Authentication filters** (responsible for the creation of the authenticated object of the Authentication implementation and setting it into the SecurityContext)
- **AuthorizationFilter** (responsible for deciding if the user can perform certain action based on the data in the Authentication)
- **Other filters**: attack-protection (e.g., CsrfFilter), exception mapping (e.g., ExceptionTranslationFilter), etc.

✅ 🌐 Vue d'ensemble des composants clés de Spring Security

Élément	Rôle
`SecurityFilterChain`	Contient tous les filtres de sécurité. C'est le cœur de Spring Security.
`Authentication`	Représente l'utilisateur authentifié.
`SecurityContext`	Stocke l'objet `Authentication` — pour connaître l'utilisateur courant.
`HttpSecurity`	Un *builder* (constructeur DSL) utilisé pour créer la `SecurityFilterChain`.

⚙️ Comment ça fonctionne concrètement ?

- On configure `HttpSecurity` dans une méthode annotée `@Bean` dans une classe `@Configuration`.
- On y utilise les méthodes comme :

```
http
  .authorizeHttpRequests()
  .anyRequest().authenticated()
  .and()
  .csrf().disable();
```

- Puis on appelle `.build()` → cela génère une `SecurityFilterChain` prête à l'emploi.
- Cette chaîne est injectée automatiquement dans le système.

✅ **Authentication interface** has 6 methods:

```
/.../

package org.springframework.security.core;

import ...

/** Represents the token for an authentication request or for an authenticated principal ...*/
public interface Authentication extends Principal, Serializable {

    /** Set by an <code>AuthenticationManager</code> to indicate the authorities that the ...*/
    Collection<? extends GrantedAuthority> getAuthorities();

    /** The credentials that prove the principal is correct. This is usually a password, ...*/
    Object getCredentials();

    /** Stores additional details about the authentication request. These might be an IP ...*/
    Object getDetails();

    /** The identity of the principal being authenticated. In the case of an authentication ...*/
    Object getPrincipal();

    /** Used to indicate to {@code AbstractSecurityInterceptor} whether it should present ...*/
    boolean isAuthenticated();

    /** See {@link #isAuthenticated()} for a full description. ...*/
    void setAuthenticated(boolean isAuthenticated) throws IllegalArgumentException;

}
```

1. `isAuthenticated()`

🟢 **À quoi ça sert ?**

→ Savoir si l'utilisateur est **déjà authentifié** (`true`) ou **pas encore** (`false`).

🔍 Utilisé par le système pour savoir :

→ Est-ce qu'on traite la requête comme un utilisateur connecté ou pas ?

---

2. `getPrincipal()`

🟢 **Qui est connecté ?**

→ Renvoie les **infos sur l'utilisateur** (objet `UserDetails` , ou même un objet custom).

📌 Quand tu mets `@AuthenticationPrincipal` dans un contrôleur, Spring appelle cette méthode.

```
@GetMapping("/me")
public String currentUser(@AuthenticationPrincipal CustomUserDetails user) {
    return user.getUsername();
}
```

---

3. `getAuthorities()`

🟢 **Quels sont ses rôles ?**

→ Retourne une liste des **rôles ou permissions** (`ROLE_USER` , `ROLE_ADMIN` , etc).

📌 Utilisé pour savoir si l'utilisateur a le droit d'accéder à une ressource (`@PreAuthorize` , etc).

4. `getCredentials()`

🟡 **Contient les infos de connexion au début (login)**

→ Par exemple, le **mot de passe entré** pendant le login.

🧠 Elle est utile **avant authentification**, pas après.

---

5. `getDetails()`

🟡 **Détails supplémentaires sur la requête**

→ Par exemple : l'adresse IP, le user agent, etc.

📌 Peu utilisé, sauf dans des cas spécifiques.

---

6. `setAuthenticated(boolean)`

🔴 **Active ou désactive le flag** `authenticated`

Mais ⚠️ **déconseillé** à utiliser directement.

✅ Mieux : recréer un nouvel objet `Authentication` avec `isAuthenticated = true` .

🔐 `Authentication` , `SecurityContext` , `SecurityContextHolder`

1. `Authentication` **= l'objet qui contient :**

- l'utilisateur (`principal`)

- ses rôles (`authorities`)
- son état (`isAuthenticated`)

🔹 Il est **créé après l'authentification réussie**, par un **filtre Spring Security**.

---

2. `SecurityContext` = une boîte qui contient l'objet `Authentication`

🧠 C'est une **boîte globale par requête** pour dire :

> "Voici l'utilisateur connecté actuellement"

---

3. `SecurityContextHolder` = accès global au `SecurityContext`

📦 Tu veux accéder à l'utilisateur courant dans ton code ?
➤ Utilise `SecurityContextHolder.getContext().getAuthentication()`

🧠 Par défaut, Spring utilise le **ThreadLocal** (1 thread = 1 utilisateur)

→ Car chaque requête HTTP est gérée dans un thread séparé (servlet), donc pas de conflit.

🔧 Tu peux changer ce comportement via :

spring.security.strategy=MODE_INHERITABLETHREADLOCAL

ou par code :

```
SecurityContextHolder.setStrategyName("MODE_GLOBAL");
```

📌 💡 Comment récupérer l'utilisateur connecté

① Dans un contrôleur avec paramètre direct :

```
@GetMapping("/home")
public String home(Authentication authentication) {
    return authentication.getName(); // ou getPrincipal()
}
```

② En utilisant `@AuthenticationPrincipal` :

```
@GetMapping("/me")
```

```
public String me(@AuthenticationPrincipal CustomUserDetails user) {
    return user.getEmail();
}
```

③ Depuis n'importe où (ex. service) :

```
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
Object principal = auth.getPrincipal();
```

> `CustomUserDetails` is **your own class** (you create it) that **implements** the Spring Security interface UserDetails.
>
> UserDetails **core interface** used by Spring Security to represent the **currently authenticated user**.

✅ Ajout de la dépendance

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

➡️ **Spring Security est activé automatiquement**
➡️ Une sécurité de base est mise en place **par défaut**.

🔐 Comportement par défaut (sans rien configurer)

- Un **formulaire de login** est activé automatiquement.
- Un utilisateur est disponible :
  - **Nom** : `user`
  - **Mot de passe** : généré automatiquement et affiché dans la console

---

⚙️ Modifier les identifiants

Dans `application.yml` ou `application.properties` :

```
spring:
  security:
    user:
      name: admin
      password: admin123
```

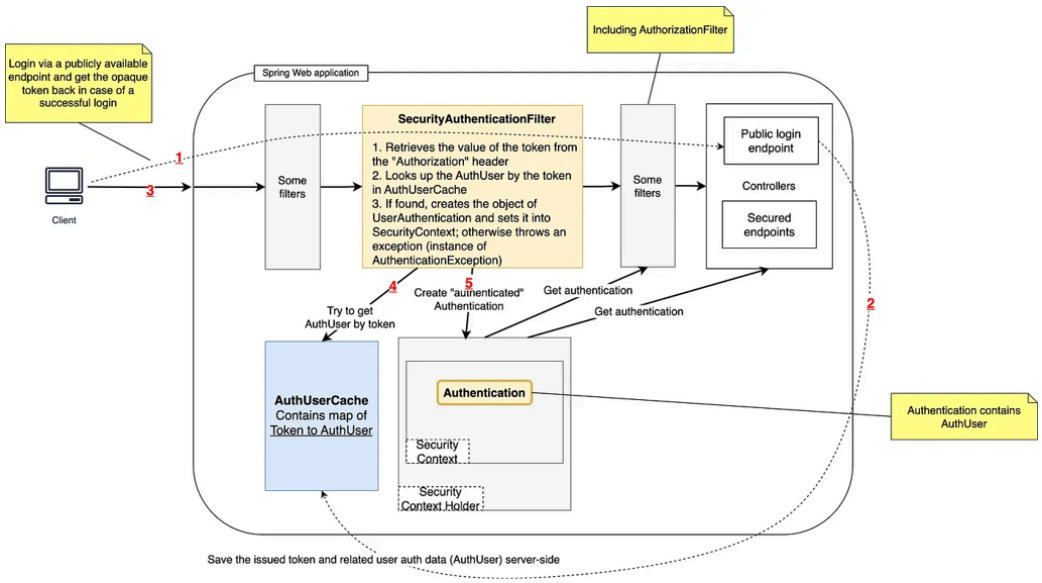
⚠️ En production, il ne faut pas garder la config par défaut

→ Car tout le monde connaît l'utilisateur par défaut.
→ Il faut créer **ta propre sécurité** (SecurityFilterChain, authentification personnalisée, etc.)

❌ Désactiver la sécurité par défaut

Spring active la sécurité par défaut **si tu ne déclares rien**.
Pour la désactiver :

- Crée **ton propre** `SecurityFilterChain` (`@Bean`)
- Ou déclare un bean `AuthenticationManager`, `AuthenticationProvider` ou `UserDetailsService`



Step 1: Creating a class to represent authenticated user data.

You create a simple class (`AuthUser`) that holds user info (e.g., ID and roles). It doesn't need to implement any interface.(put any attributes you need there)

```
public record AuthUser(String userId, List<Role> roles) {}
```

Step 2: Creating a custom implementation of the Authentication interface.

You create `UserAuthentication`, which wraps your `AuthUser` and implements `Authentication`.

```
public record UserAuthentication(AuthUser authUser) implements Authentication {
 @Override
 public Collection<? extends GrantedAuthority> getAuthorities() {
   return authUser.roles().stream()
       .map(Enum::name)
       .map(SimpleGrantedAuthority::new)
       .collect(Collectors.toSet());
 }
 @Override
 public Object getCredentials() {
   return null;
```

```java
  }
  @Override
  public Object getDetails() {
    return null;
  }
  @Override
  public Object getPrincipal() {
    return authUser;
  }
  @Override
  public boolean isAuthenticated() {
    // This value is set to true in this example because Authentication is used only to represent
    // an authenticated user and not for transferring authentication details
    return true;
  }
  @Override
  public void setAuthenticated(boolean isAuthenticated) throws IllegalArgumentException {
    // NoOp
  }
  @Override
  public String getName() {
    return null;
  }
}
```

You **override these methods** because Spring Security only understands objects that implement the `Authentication` interface. By doing this:

- You control what user info is available.
- You make the framework authorize users properly.
- You ensure compatibility with `SecurityContextHolder`.

Method "**getAuthorities**()" returns a collection of type GrantedAuthority representing roles of the user to be evaluated by the authorization part of the Spring Security framework.

Remember, that, by default, SpringSecurity expects role names to have a "**ROLE_**" prefix. Otherwise, authorization will ignore such roles. You can override the role prefix by registering the bean of type

Step 3: Login/Logout functionality implementation

The login method:

- Gets user by username
- Verifies password
- Generates a token

- Stores the token with user data in a simple in-memory map (`AuthUserCache`)
- Returns the token to the client

The logout method:

- Deletes the token from cache

```java
@RestController
@RequestMapping("/api/auth")
public class AuthApiController {
 private final AuthService authService;

 public AuthApiController(AuthService authService) {
   this.authService = authService;
 }

 @PreAuthorize("isAnonymous()")
 @PostMapping("/login")
 public TokenDto login(@RequestBody LoginDto loginDto) {
   return authService.login(loginDto);
 }

 @PreAuthorize("isAuthenticated()")
 @PostMapping("/logout")
 @SecurityRequirement(name = OpenApiConstants.TOKEN_SECURITY_REQUIREMENT)
 public void logout(HttpServletRequest httpServletRequest) {
   String token =
       Optional.ofNullable(httpServletRequest.getHeader(AuthConstants.AUTHORIZATION_HEADER))
           .orElseThrow();


   authService.logout(token);
 }
}
```

```java
@Service
public class AuthService {
 private final AuthUserCache authUserCache;
 private final UserService userService;
 private final PasswordEncoder passwordEncoder;


 public AuthService(
     AuthUserCache authUserCache, UserService userService, PasswordEncoder passwordEncoder) {
   this.authUserCache = authUserCache;
   this.userService = userService;
   this.passwordEncoder = passwordEncoder;
 }


 public TokenDto login(LoginDto loginDto) {
   UserResponseWithCredentials userCredentials =
       userService.getUserCredentialsByUsername(loginDto.username());
   if (!passwordEncoder.matches(loginDto.password(), userCredentials.passwordHash())) {
     throw new ApplicationAuthenticationException("Password is incorrect");
   }
```

```
  String token = UUID.randomUUID().toString();
  UserResponse userResponse = userCredentials.userResponse();
  AuthUser authUser = new AuthUser(userResponse.id(), userResponse.roles());
  authUserCache.login(token, authUser);
  return new TokenDto(token);
 }


 public void logout(String token) {
  authUserCache.logout(token);
 }
}
```

🔐 **Connexion et gestion des sessions :**

- Quand un utilisateur se connecte, on génère un **token aléatoire** (`UUID`) et on l'associe à l'utilisateur.
- Ce token est stocké dans une **Map en mémoire** (`Map<String, AuthUser>`) → cela garde les utilisateurs connectés.

⚠️ **Problème :**
Si on déploie plusieurs instances de l'application (avec un load balancer), **chaque instance aura sa propre mémoire** → les utilisateurs ne seront pas reconnus sur les autres instances.

✅ **Solutions en production :**

1. **Redis** : utiliser un **cache centralisé** accessible par toutes les instances.
2. **Redis Pub/Sub** : synchroniser les caches internes entre les instances.
3. **Sticky sessions** : le load balancer envoie toujours un utilisateur à **la même instance** (grâce à un cookie ou IP).

```
// Decentralized cache (e.g. Redis) or key-value database will be, most likely, used in production
// scenario
@Component
public class AuthUserCache {
 // You may want to use some collection with time-to-leave to handle session expiration
 private final Map<String, AuthUser> sessions = new ConcurrentHashMap<>();
 public void login(String token, AuthUser authUser) {
   sessions.put(token, authUser);
 }
 public void logout(String token) {
   sessions.remove(token);
 }
 public Optional<AuthUser> getByToken(String token) {
```

```
    return Optional.ofNullable(sessions.get(token));
  }
}
```

We define **BCryptPasswordEncoder** bean (in SecurityConfig) as an implementation of the PasswordEncoder for the application.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    // ... SecurityFilterChain or other beans
}

```

Pourquoi ?

Spring Security a besoin d'un `PasswordEncoder` pour vérifier si le mot de passe entré correspond au hash enregistré.

We also need to implement the method in UserService to get the user by username, because that is how login is implemented in our case: get user by username → if found, compare the hash of the password from login request to the one, saved in the database.

```
@Service
public class UserService {

    private final UserRepository userRepository;
    private final UserMapper userMapper; // Pour transformer UserEntity en UserResponse

    public UserService(UserRepository userRepository, UserMapper userMapper) {
        this.userRepository = userRepository;
        this.userMapper = userMapper;
    }

    public UserResponseWithCredentials getUserCredentialsByUsername(String username) {
        UserEntity userEntity = userRepository.findByUsername(username)
                .orElseThrow(() -> new NotFoundException("User not found"));

        return new UserResponseWithCredentials(
            userMapper.toResponse(userEntity),
            userEntity.getPasswordHash()
        );
    }
}
```

Step 4: Custom security filter implementation.

Implement a filter responsible for authentication of each request, that is coming to the application. We do it by creating a class (annotated as a Spring Component, so we can inject it later in our SecurityConfig class) that extends OncePerRequestFilter.

```java
@Component
public class SecurityAuthenticationFilter extends OncePerRequestFilter {

  private final AuthUserCache authUserCache;

  public SecurityAuthenticationFilter(AuthUserCache authUserCache) {
    this.authUserCache = authUserCache;
  }

  @Override
  protected void doFilterInternal(
      HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
      throws ServletException, IOException {

    String authenticationHeader = request.getHeader(AuthConstants.AUTHORIZATION_HEADER);

    if (authenticationHeader == null) {
      // Authentication token is not present, let's rely on anonymous authentication
      filterChain.doFilter(request, response);
      return;
    }

    AuthUser authUser =
        authUserCache
            .getByToken(authenticationHeader)
            .orElseThrow(() -> new TokenAuthenticationException("Token is not valid"));

    UserAuthentication authentication = new UserAuthentication(authUser);

    SecurityContext securityContext = SecurityContextHolder.createEmptyContext();
    securityContext.setAuthentication(authentication);
    SecurityContextHolder.setContext(securityContext);

    filterChain.doFilter(request, response);
  }
}
```

The job of the filter is pretty simple: to fetch the token from the header and then to try to get the AuthUser from the cache by that token. If the user is found in the cache, it means that the authentication is successful, so the filter creates a UserAuthentication object and sets it to the SecurityContext (otherwise an instance of AuthenticationException is thrown).

Step 5: Exception handling.

Par défaut, Spring Security retourne des messages d'erreur simples quand :

- L'utilisateur **n'est pas authentifié** (`401 Unauthorized`)
- L'utilisateur **n'a pas les droits** (`403 Forbidden`)

Mais ces messages ne sont pas très jolis ou personnalisés. Donc on va créer nos propres gestionnaires d'erreurs pour retourner des réponses JSON claires.

1. AuthenticationEntryPoint

- Gère les erreurs **d'authentification** (ex: token manquant ou invalide)

Tu dois l'implémenter pour retourner une erreur personnalisée `401`.

```
@Component
public class ApplicationAuthenticationEntryPoint implements AuthenticationEntryPoint {
    private final ObjectMapper objectMapper;

    public ApplicationAuthenticationEntryPoint(ObjectMapper objectMapper) {
        this.objectMapper = objectMapper;
    }

    @Override
    public void commence(HttpServletRequest request, HttpServletResponse response,
                         AuthenticationException authException) throws IOException {
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        response.setContentType("application/json");
        objectMapper.writeValue(response.getOutputStream(),
            new ApiErrorResponse(authException.getMessage()));
    }
}
```

2. AccessDeniedHandler

- Gère les erreurs **d'autorisation** (ex: l'utilisateur est connecté, mais n'a pas accès)

Retourne une erreur `403`.

```
@Component
public class ApplicationAccessDeniedHandler implements AccessDeniedHandler {
    private final ObjectMapper objectMapper;

    public ApplicationAccessDeniedHandler(ObjectMapper objectMapper) {
        this.objectMapper = objectMapper;
    }

    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response,
                       AccessDeniedException accessDeniedException) throws IOException {
        response.setStatus(HttpServletResponse.SC_FORBIDDEN);
        response.setContentType("application/json");
        objectMapper.writeValue(response.getOutputStream(),
            new ApiErrorResponse(accessDeniedException.getMessage()));
    }
}
```

🚨 Exceptions personnalisées

Tu peux créer tes propres exceptions pour contrôler les erreurs d'authentification.

```java
public class ApplicationAuthenticationException extends AuthenticationException {
    public ApplicationAuthenticationException(String msg) {
        super(msg);
    }
}
public class TokenAuthenticationException extends AuthenticationException {
    public TokenAuthenticationException(String message) {
        super(message);
    }
}

```

- ✅ **ExceptionTranslationFilter** (de Spring Security) s'occupe de rediriger vers les bons handlers :
  - `AuthenticationEntryPoint` si erreur d'authentification.
  - `AccessDeniedHandler` si erreur d'autorisation.
- ❌ **@ControllerAdvice ne fonctionne pas** avec les exceptions lancées depuis les **filtres** (`Filter`).
  - Car l'ordre d'exécution est : **Filters** → **DispatcherServlet** → **ControllerAdvice**
- ✅ Les exceptions lancées dans des contrôleurs ou via `@PreAuthorize` passent bien par `@ControllerAdvice`.

Élément	Utilité
`AuthenticationEntryPoint`	Personnaliser erreur 401 (non connecté)
`AccessDeniedHandler`	Personnaliser erreur 403 (accès refusé)
Exceptions custom	Pour contrôler tes propres messages
`ExceptionTranslationFilter`	Redirige vers les bons handlers
`@ControllerAdvice`	Ne gère **pas** les exceptions des filtres

Step 6: Security configuration.

```java
@EnableMethodSecurity // allow to specify access via annotations
@Configuration
public class SecurityConfig {

  private final SecurityAuthenticationFilter securityAuthenticationFilter;

  private final AuthenticationEntryPoint authenticationEntryPoint;

  private final AccessDeniedHandler accessDeniedHandler;

  public SecurityConfig(
      SecurityAuthenticationFilter securityAuthenticationFilter,
      AuthenticationEntryPoint authenticationEntryPoint,
      AccessDeniedHandler accessDeniedHandler) {

    this.securityAuthenticationFilter = securityAuthenticationFilter;
    this.authenticationEntryPoint = authenticationEntryPoint;
    this.accessDeniedHandler = accessDeniedHandler;
  }

  @Bean
  public PasswordEncoder encoder() {
    return new BCryptPasswordEncoder();
  }

  @Bean
  public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

    http.addFilterBefore(securityAuthenticationFilter, AuthorizationFilter.class)
        .authorizeHttpRequests(
            mather ->
                mather
                    .requestMatchers(
                        "/swagger-ui.html",
                        "/swagger-ui/*",
                        "/v3/api-docs",
                        "/v3/api-docs/swagger-config")
                    .permitAll())
        .authorizeHttpRequests(
            matcher ->
                matcher
                    // method security will be evaluated after DSL configs,
                    // so we have to define public paths upfront
                    .requestMatchers(HttpMethod.POST, "/api/auth/login", "/api/users")
                    .permitAll())
        .authorizeHttpRequests(matcher -> matcher.anyRequest().authenticated())
        .csrf(AbstractHttpConfigurer::disable)
        .sessionManagement(
            configurer -> configurer.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .exceptionHandling(
            customizer ->
                customizer
                    .accessDeniedHandler(accessDeniedHandler)
                    .authenticationEntryPoint(authenticationEntryPoint));

    return http.build();
  }

```

```
  // register NoOp AuthenticationManager to avoid log printed by default autoconfiguration
  @Bean
  public AuthenticationManager noOpAuthenticationManager() {
    return authentication -> null;
  }
}
```

1. **Annotation** `@EnableMethodSecurity`

> Active l'utilisation des annotations comme `@PreAuthorize("hasRole('ADMIN')")`.

2. **Bean** `SecurityFilterChain`

C'est ici qu'on :

- ajoute **ton filtre custom** `SecurityAuthenticationFilter`
- définit les **routes publiques** (`/login`, `/swagger-ui`, etc.)
- rend toutes les autres **routes protégées**
- **désactive CSRF** (inutile avec token)
- utilise une **politique stateless** (pas de session serveur)
- branche **tes handlers d'erreurs personnalisés**

3. **Bean** `PasswordEncoder`

> Utilisé pour encoder/vérifier les mots de passe (ici avec BCrypt)

4. **Bean** `AuthenticationManager` **(No-op)**

> On le fournit juste pour empêcher Spring de créer sa propre config et d'afficher des logs inutiles.

❌ 🔁 **Changements importants dans Spring Security avec Spring Boot 3+**

1. ❌ `WebSecurityConfigurerAdapter` **est supprimé**
   - **Avant (2.x)** : on héritait de `WebSecurityConfigurerAdapter` pour configurer la sécurité.
   - **Maintenant (3.x)** : on utilise un **bean de type** `SecurityFilterChain` à la place.

2. ✅ `HttpSecurity` est maintenant **chaînable avec lambdas** (`authorizeHttpRequests(...)`)
3. ✅ `@EnableWebSecurity` n'est plus obligatoire
4. ✅ `PasswordEncoder` reste identique, mais tu dois **le déclarer comme un** `@Bean`
5. ✅ Le système d'**authentification personnalisée** se base sur `AuthenticationManager` **exposé en tant que bean**
6. ✅ `UserDetailsService` doit être déclaré si tu fais ton propre login

Étape	Description
1	`AuthUser` → Représente un utilisateur connecté
2	`UserAuthentication` → Classe de sécurité qui encapsule `AuthUser`
3	`AuthService` + `AuthApiController` → Gère login/logout
4	`AuthUserCache` → Stocke les tokens en mémoire
5	`SecurityAuthenticationFilter` → Authentifie chaque requête
6	`AuthenticationEntryPoint` / `AccessDeniedHandler` → Gèrent les erreurs 401/403
7	`SecurityConfig` → Configure les routes sécurisées,

	filtre, erreurs